

Machine Learning

Week 2 Lecture

Dr. Muhammad Ahmed Raza

Contents

- Linear Regression
- Polynomial Regression
- Underfitting vs. Overfitting
- Bias Variance Trade off
- Regularization

THE OBJECTIVE: MEAN SQUARED ERROR (MSE)

Defining success by minimizing the difference between prediction and reality.

The Difference

Calculates error for a single data point.

The Square

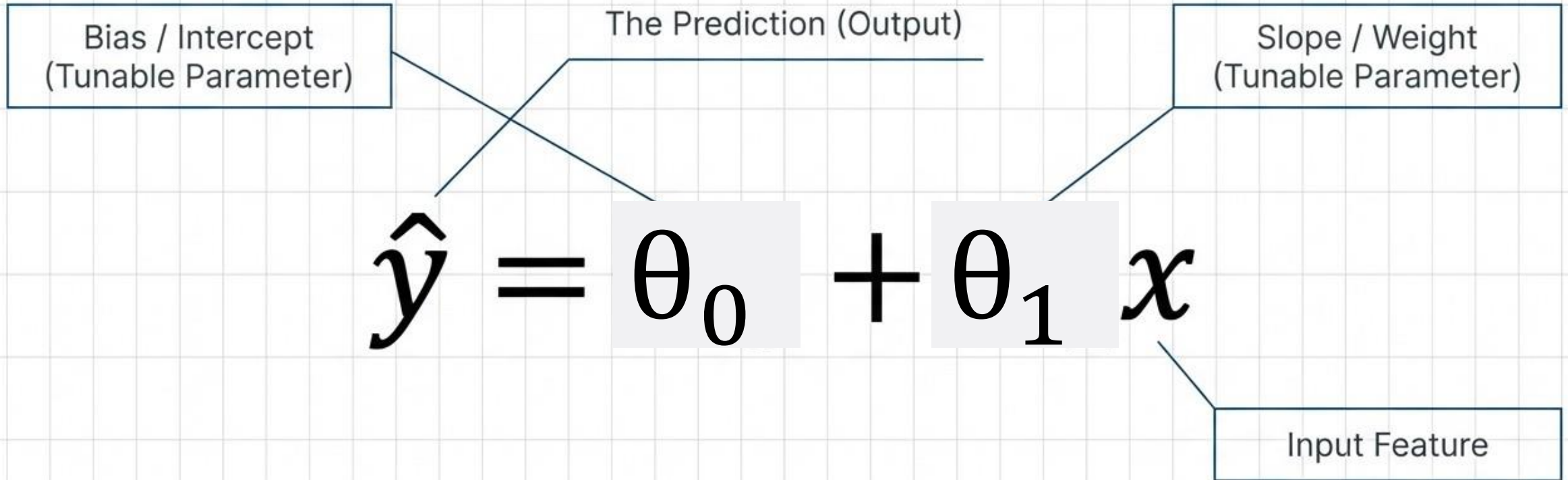
Penalizes large outliers heavily and ensures positive values.

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The Mean

Averages the error across the entire dataset.

ARCHITECTURE: THE SINGLE-FEATURE MODEL



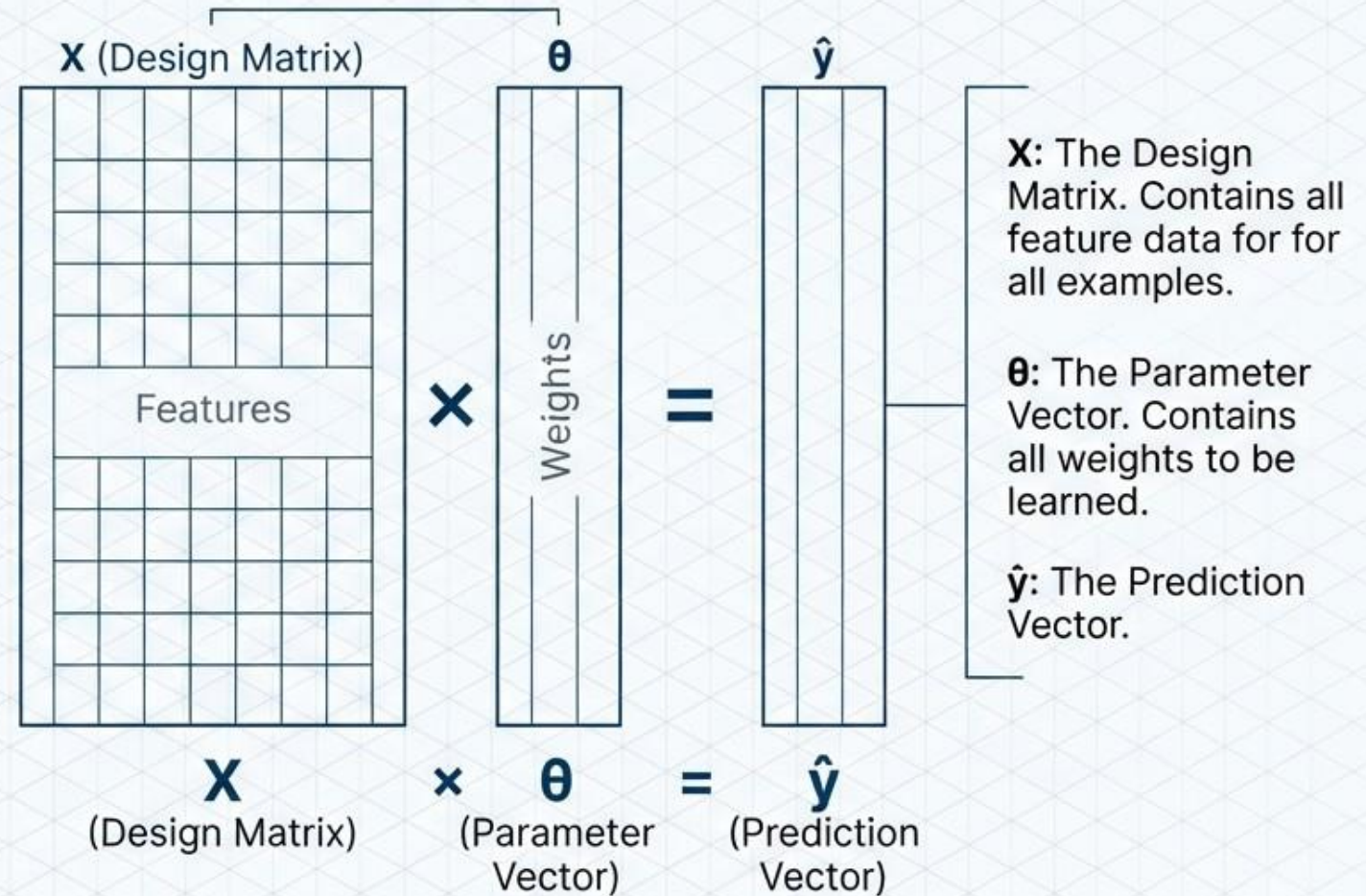
System definition for a single variable.
 w_0 and w_1 are the knobs we turn to calibrate the machine.

SCALING THE ARCHITECTURE: MATRIX NOTATION

Vectorizing operations for multi-feature systems.

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$$

Efficient computation for
n-dimensional data.



EXECUTION: TWO WAYS TO SOLVE

Method 1: Optimization
(**Gradient Descent**)



Iterative Approach.



Method 2: Closed-Form
(**Normal Equation**)

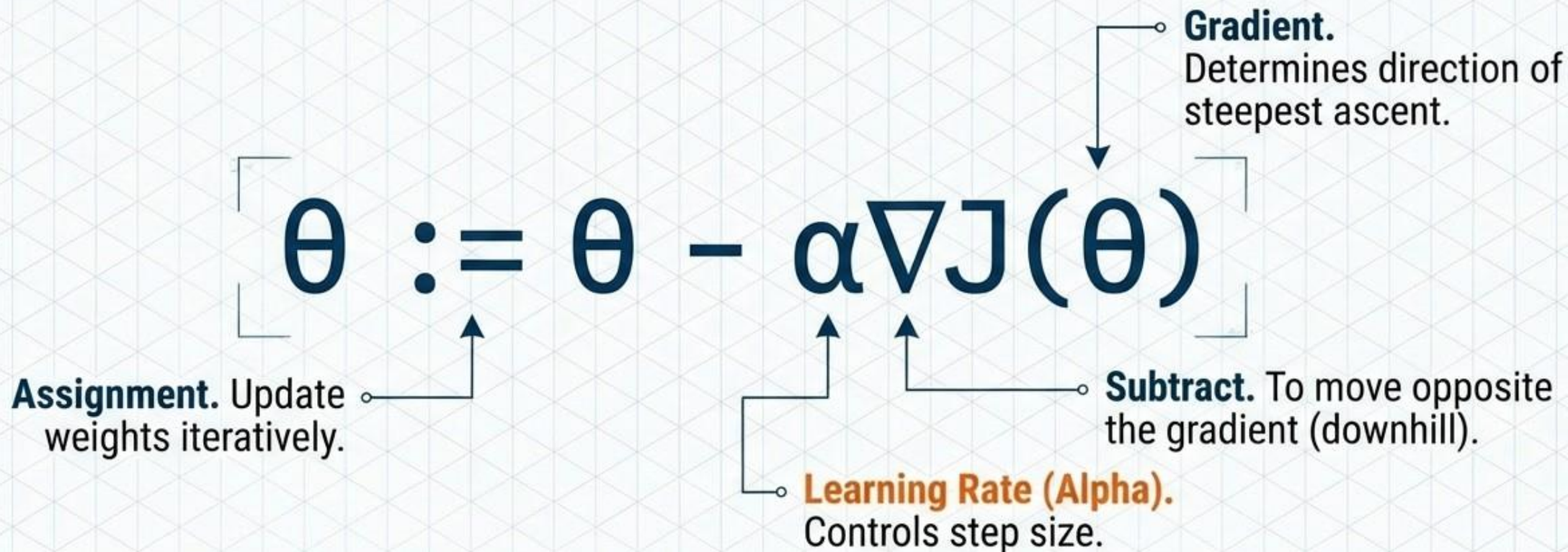


Analytical Approach.

We can either **iterate** our way to the solution (**numerical**)
or **calculate it directly** using linear algebra (**analytical**).

METHOD 1: GRADIENT DESCENT (OPTIMIZATION)

The iterative algorithm that “walks” downhill.



Industry Standard. Used in almost all modern ML systems.
Scales efficiently to large datasets.

METHOD 2: THE NORMAL EQUATION (CLOSED-FORM)

A one-shot analytical solution.

$$\theta = (X^T X)^{-1} X^T y$$

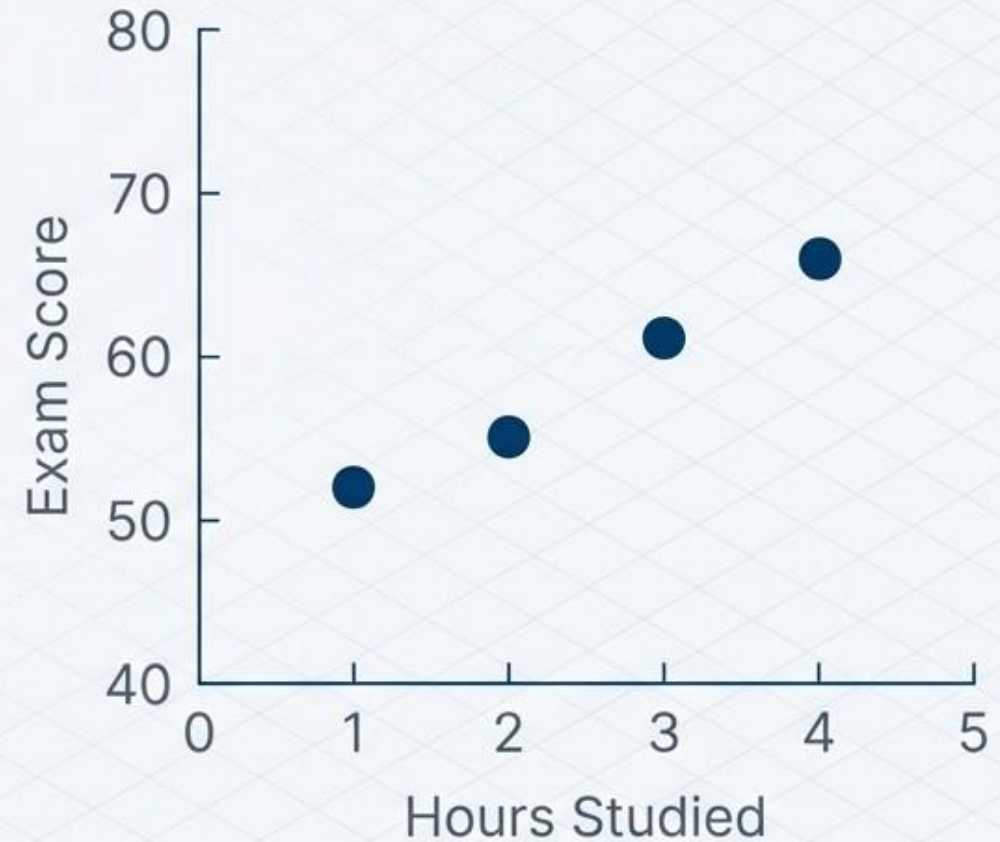
**MATRIX INVERSION
REQUIRED**



- **Pros:** Exact solution in one step. No learning rate needed.
- **Cons:** Computationally expensive $O(n^3)$. Unstable for massive feature sets.

CASE STUDY: EXAM SCORE PREDICTION

Hours (x)	Score (y)
1	52
2	55
3	61
4	66



CONFIGURING THE SYSTEM

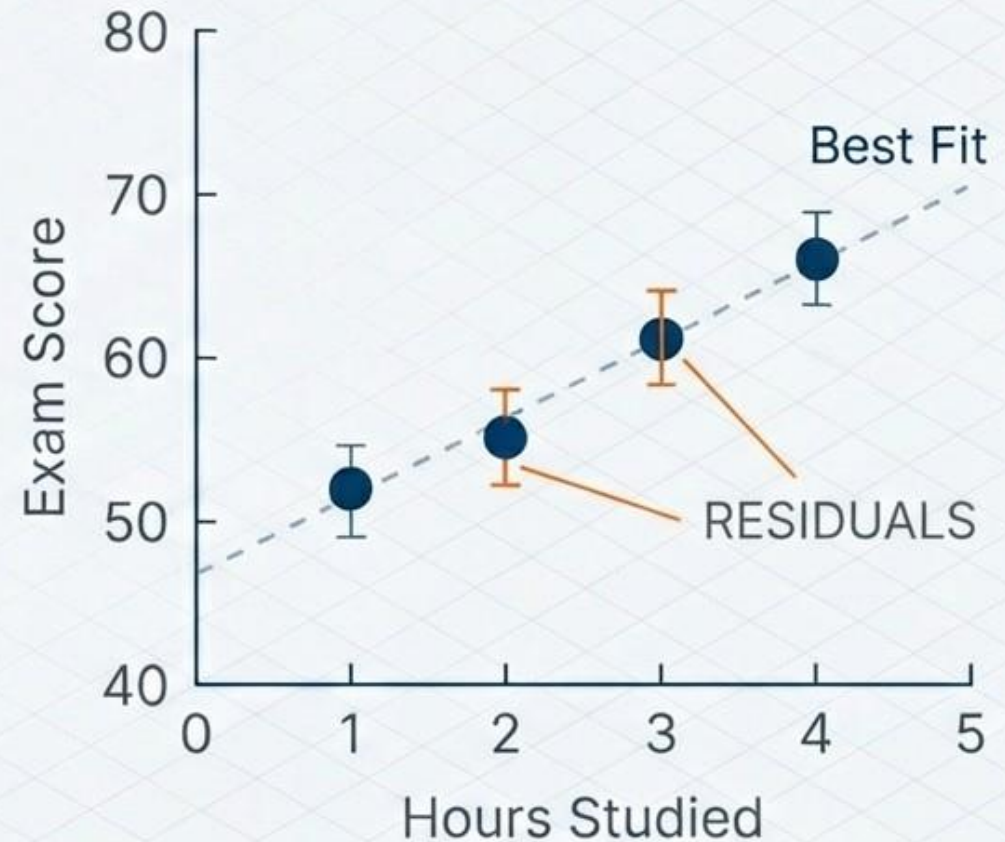
STEP 1: MODEL DEFINITION

$$\hat{y} = w_0 + w_1 x$$

STEP 2: SPECIFIC LOSS FUNCTION

$$J(w_0, w_1) = \frac{1}{4} \sum_{i=1}^n (y - (w_0 + w_1 x))^2$$

Note: $n=4$ (sample size)



SUMMARY SPECIFICATIONS

THE MODEL

$$\hat{y} = X\theta$$

- Linear in Parameters.
- X = Design Matrix.

THE OBJECTIVE

Minimize MSE.

$$J(\theta) = \frac{1}{n} \sum (\text{Error})^2$$

Convex Geometry =
Global Minimum.

THE SOLVERS

Gradient Descent:

$$\theta := \theta - \alpha \nabla J$$

(Iterative, Scalable)

Normal Equation:

$$\theta = (X^T X)^{-1} X^T y$$

(Exact, Expensive)

ENGINEERING TRADE-OFFS

Feature	Gradient Descent	Normal Equation
Type	Iterative Loop	Direct Calculation
Scalability	Excellent (Large N)	Poor (Slow Inversion)
Tuning	Requires Alpha (α)	No Hyperparameters
Use Case	Deep Learning / Big Data	Small, Simple Datasets

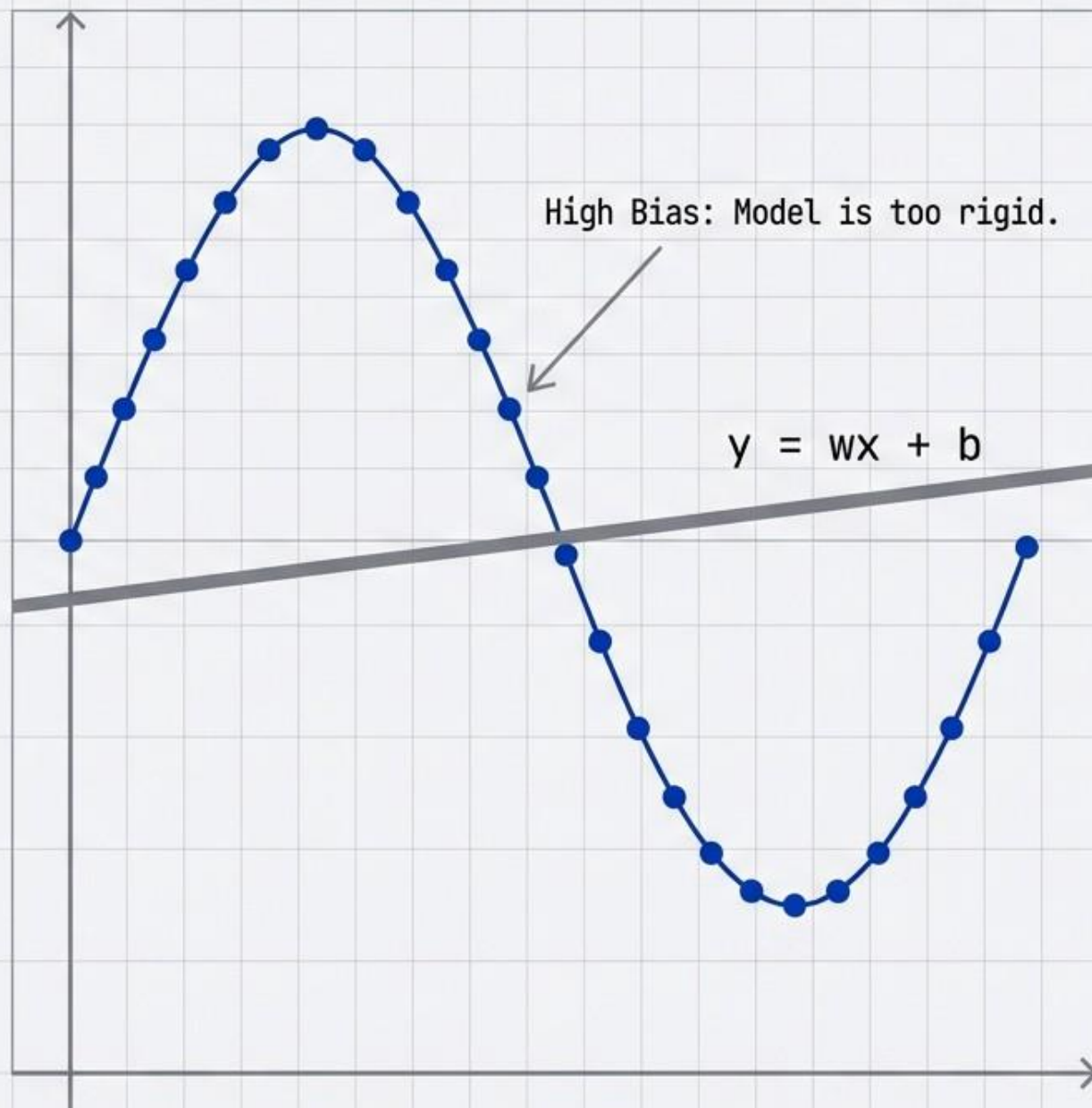
Real World Data isn't Straight

The 'Linear' Lie

Linear Regression is computationally efficient but suffers from Low Capacity. If your data represents stock prices, biological systems, or physics, it likely curves.

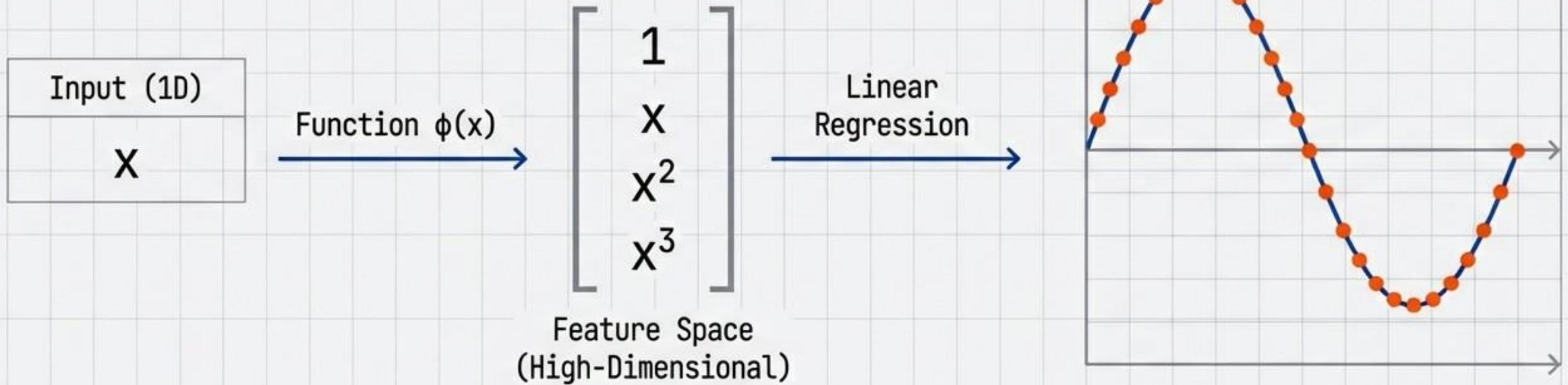
The Diagnosis: A model that assumes linearity when reality is complex is suffering from High Bias (Underfitting).

The Fix: We don't need a new algorithm; we need a better data representation.



Refactoring the Input: Polynomial Regression

Basis Expansion Strategy



The Hack: We map a low-dimensional problem into a high-dimensional feature space.

Technique: We treat x^2 and x^3 as just 'new features.' This allows us to use standard Linear Regression to solve for curved boundaries.

Analogy: The model transforms from a stiff rod into a flexible chain.

SYSTEM CONSTRAINT: THE LINEARITY PRINCIPLE

KEY IDEA:
The model must be linear in
PARAMETERS, not necessarily inputs.

$$y = \theta_0 + \theta_1 x^2$$

VALID

Linear in θ . We can transform inputs (x^2) without breaking the model.

$$y = \left(\frac{\theta_0}{\theta_1} \right) * x$$

INVALID

Non-linear relationship between parameters. This breaks the Linear Regression architecture.

Important Considerations

- However, there will be important considerations in polynomial regression:
 - Order of the polynomial model
 - Model-building strategy
 - Extrapolation
 - Ill-conditioning
 - Hierarchy

Order

- Remember:
 - it is always possible to fit a polynomial model of order $n - 1$ perfectly to a data set n points (however, this will almost surely be **overfitting!!!**)
 - One should always maintain a sense of parsimony, that is, use the simplest possible model that is consistent with the data and knowledge of the problem environment.

Model Building Strategy

- There are two standard procedures for building a polynomial model:
 - Forward selection: Successively fit models of increasing order until the t test for the highest order term is nonsignificant.
 - Backward elimination: Appropriately fit the highest order model and then delete terms one at a time, starting with the highest order, until the highest order remaining term has a significant t statistic
- Interestingly, these two procedures do not necessarily lead to the same model.

Extrapolation

- Because polynomial models may turn in unanticipated and inappropriate directions, extrapolation with them can be extremely hazardous.

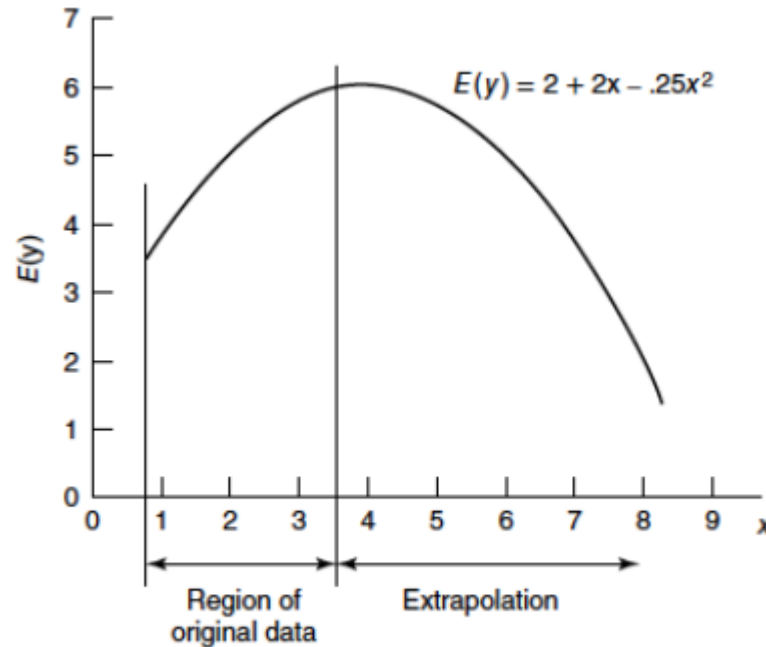


Figure 7.2 Danger of extrapolation.

Hierarchy

- The regression model:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2^2 + \theta_3 x_3^3 + \epsilon$$

Is said to be hierarchical since contains all terms of order 3 and lower. In contrast, the models:

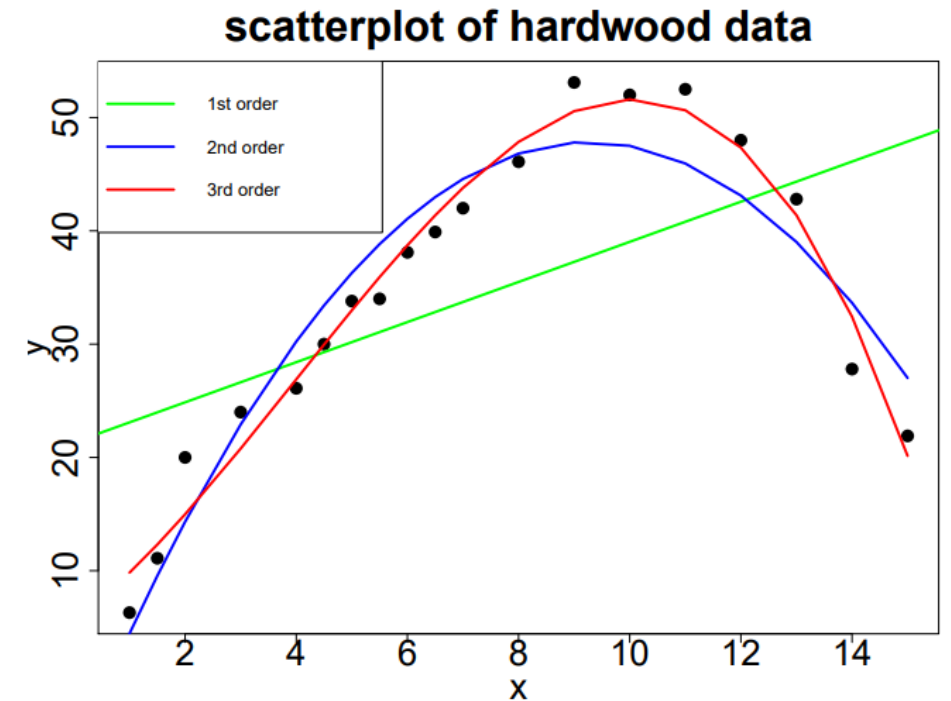
$$\hat{y} = \theta_0 + \theta_1 x + \theta_3 x_3^3 + \epsilon$$

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_1 x_2 + \epsilon$$

Are not hierarchical.

The Hardwood dataset

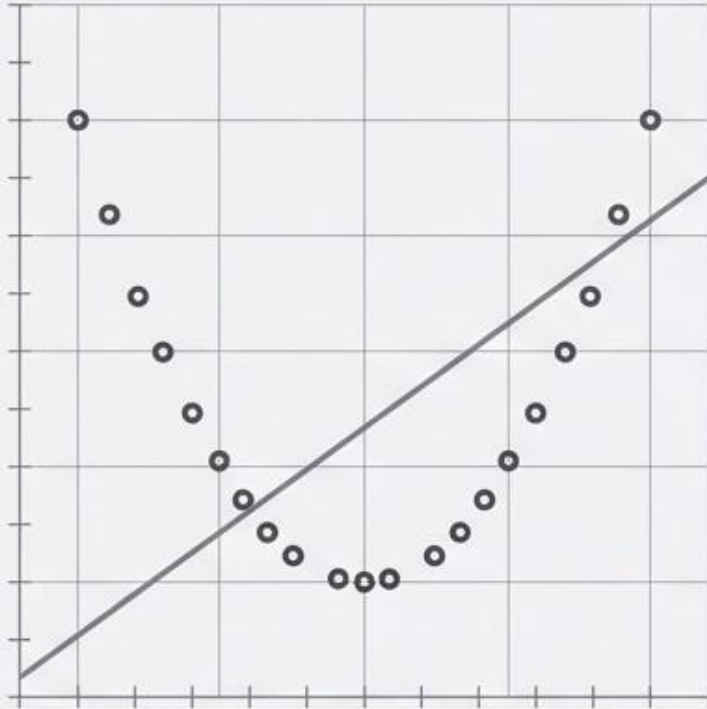
- We have 19 observations concerning
 - the strength of kraft paper (y) and
 - the percentage of hardwood (x) in the batch of pulpfrom which the paper was produced.



Week 2: Regression & Capacity Control

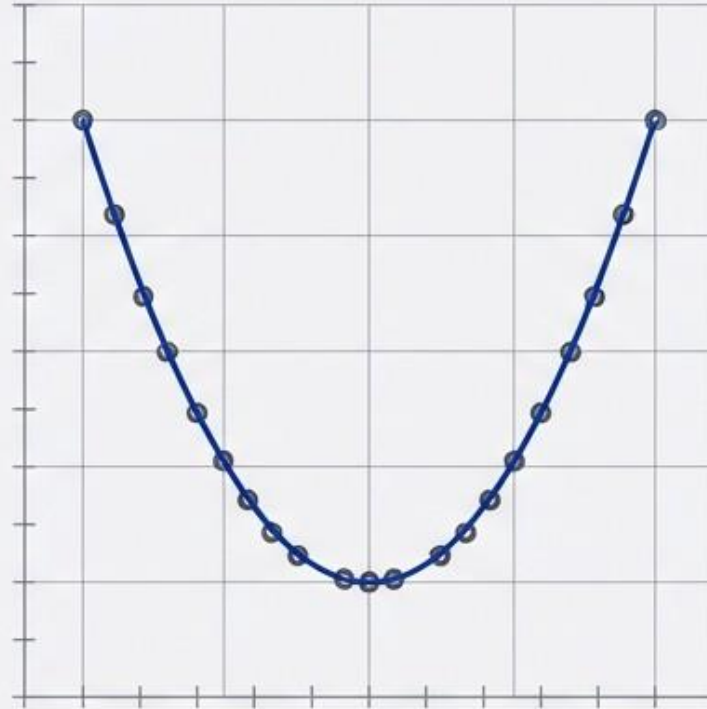
From Optimization (Training) to Generalization (Testing)

Panel 1 (Left)



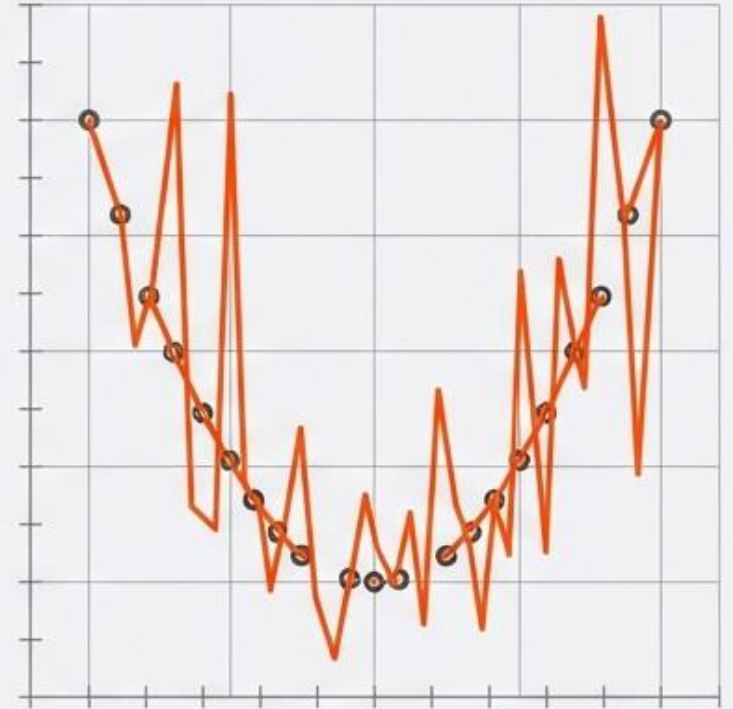
v1. Underfitting (Rigid)

Panel 2 (Center)



v2. Ideal Fit (Generalized)

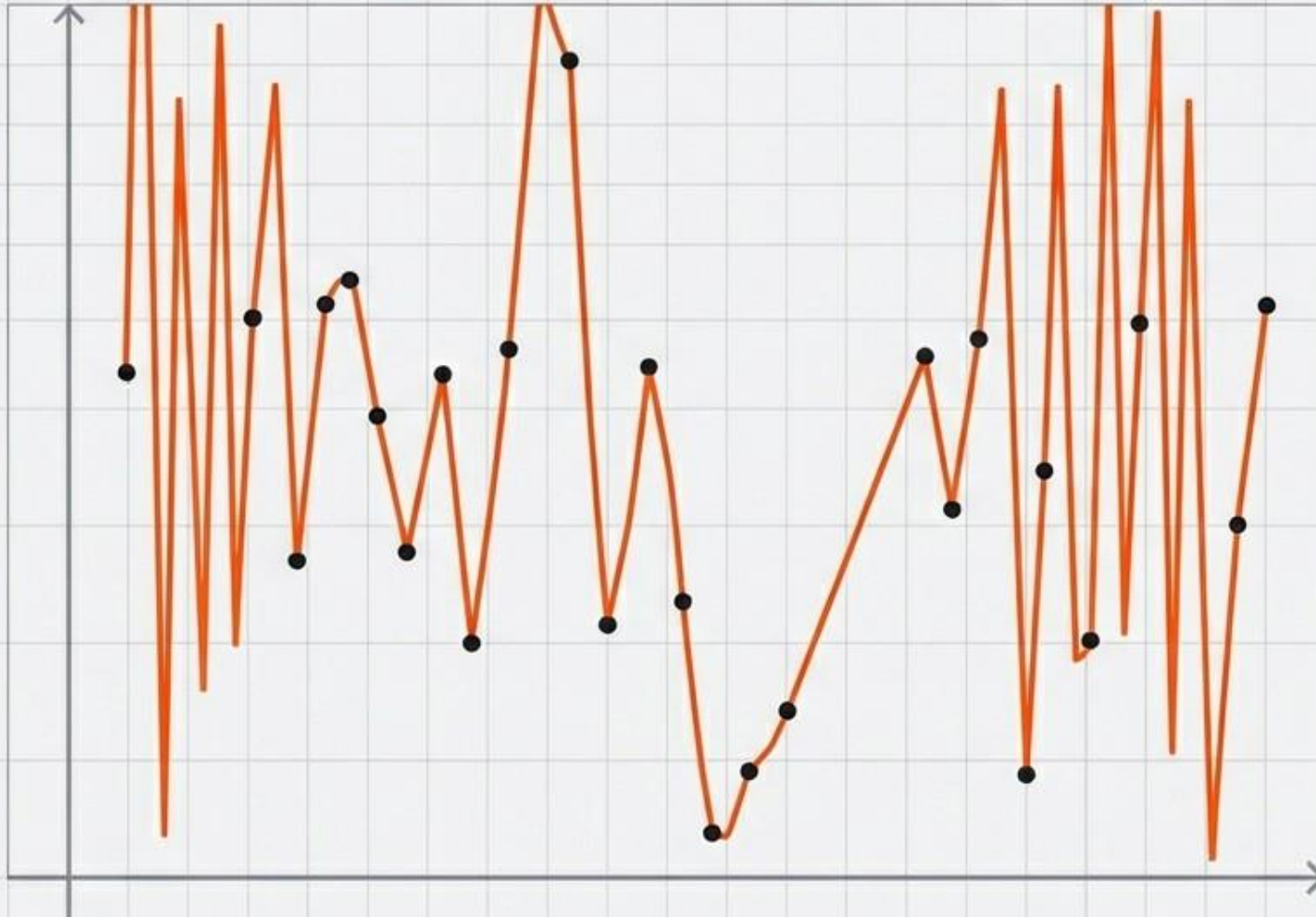
Panel 3 (Right)



v3. Overfitting (Hallucination)

The Danger of Infinite Capacity

Overfitting (High Variance)



Degree: 20

Training Error: 0.00 (Perfect)

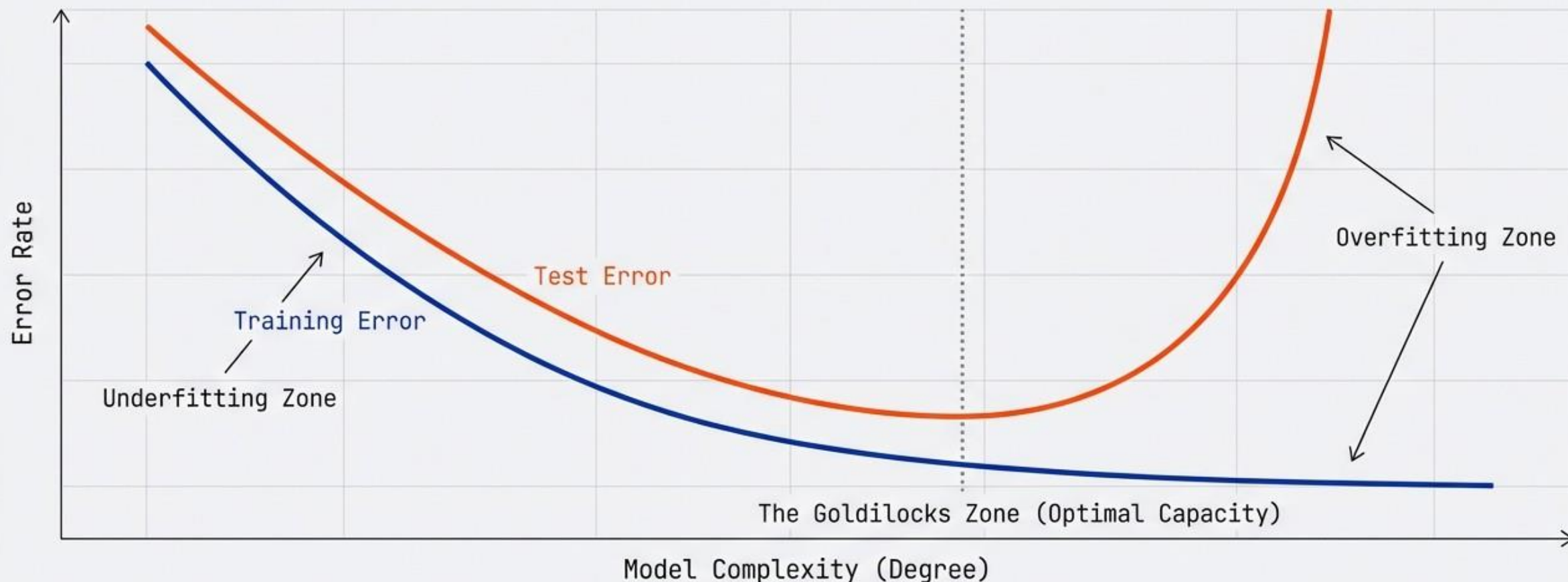
Generalization: Terrible

Diagnosis: The model has memorized the noise instead of the signal.

Engineering Equivalent: Hard-coding edge cases instead of writing general logic.

Optimization vs. Generalization

Finding the Sweet Spot



Optimization drives the **Blue Line** down.

Generalization is the art of stopping at the bottom of the **Red U-curve**.

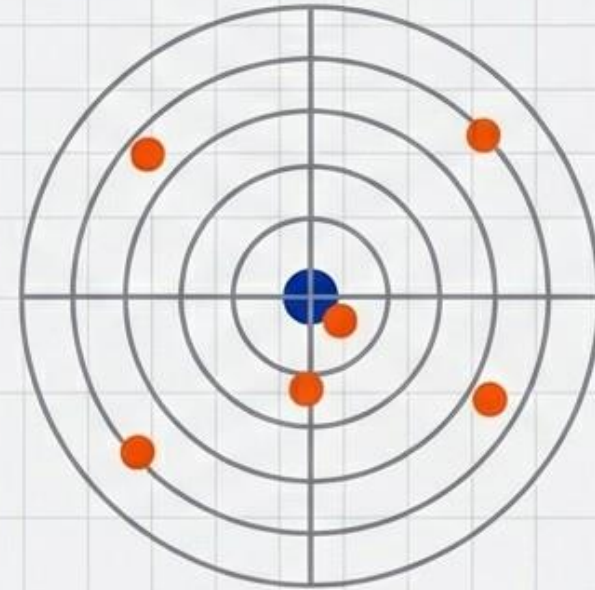
The Bias-Variance Decomposition

High Bias (Consistent, but Wrong)



Caused by: Erroneous assumptions (e.g., assuming data is linear).

High Variance (Accurate on average, but Unstable)



Caused by: Sensitivity to small fluctuations in training data.

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$$

Note: You cannot eliminate error; you can only trade bias for variance.

Bias Variance Trade off

- Mathematical Derivation

Start with target model:

Assume target $y = f(x) + \varepsilon$, with noise ε such that $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma_\varepsilon^2$.

Goal: expected squared error of estimator $\hat{f}(x)$:

$$\text{Err}(x) = \mathbb{E}_{\mathcal{D}, \varepsilon} \left[(y - \hat{f}(x))^2 \right]$$

Derivation steps (show algebra):

$$\begin{aligned} \text{Err}(x) &= \mathbb{E} \left[(f(x) + \varepsilon - \hat{f}(x))^2 \right] \\ &= \mathbb{E} \left[(f(x) - \mathbb{E}[\hat{f}(x)] + \mathbb{E}[\hat{f}(x)] - \hat{f}(x) + \varepsilon)^2 \right] \\ &= \underbrace{(f(x) - \mathbb{E}[\hat{f}(x)])^2}_{(\text{Bias})^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{Variance}} + \underbrace{\mathbb{E}[\varepsilon^2]}_{\sigma_\varepsilon^2} \end{aligned}$$

Shackling the Weights

Regularization as a Constraint

Minimize: $\text{MSE} + \lambda \cdot \text{penalty}$



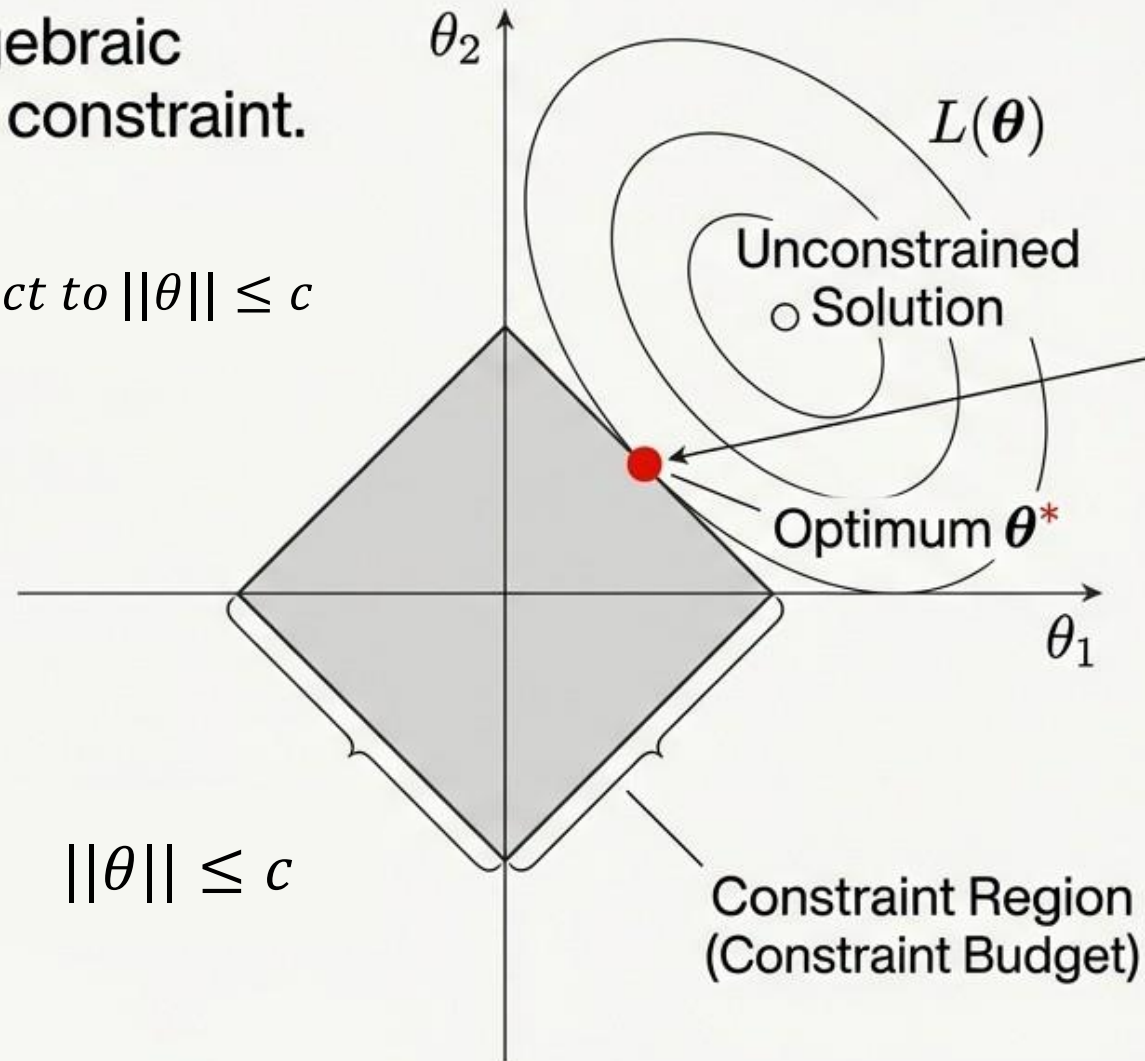
- **The Concept:** We change the rules of the game to create a 'budget' for complexity.
- **The Instruction:** 'Minimize error, BUT keep the weights small.'

- **Why?** Small weights = smoother curves = less sensitivity to noise.
- **The Knob (λ):** Controls the constraint strength. If λ is too high, the model learns nothing (Bias). If λ is 0, we risk overfitting (Variance).

The Geometry of Constraints

We can reframe the algebraic penalty as a geometric constraint.

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad \text{subject to } \|\theta\| \leq c$$



The solution is found where the loss function contours first make contact with the constraint budget.

Choosing Your Tool: L1 vs. L2



Ridge (L2)

Role: The Dampener

Outcome: Small, non-zero weights.

Best For: Numerical stability, correlated features, keeping all info.

Verdict: DEFAULT. Start here.



Lasso (L1)

Role: The Filter

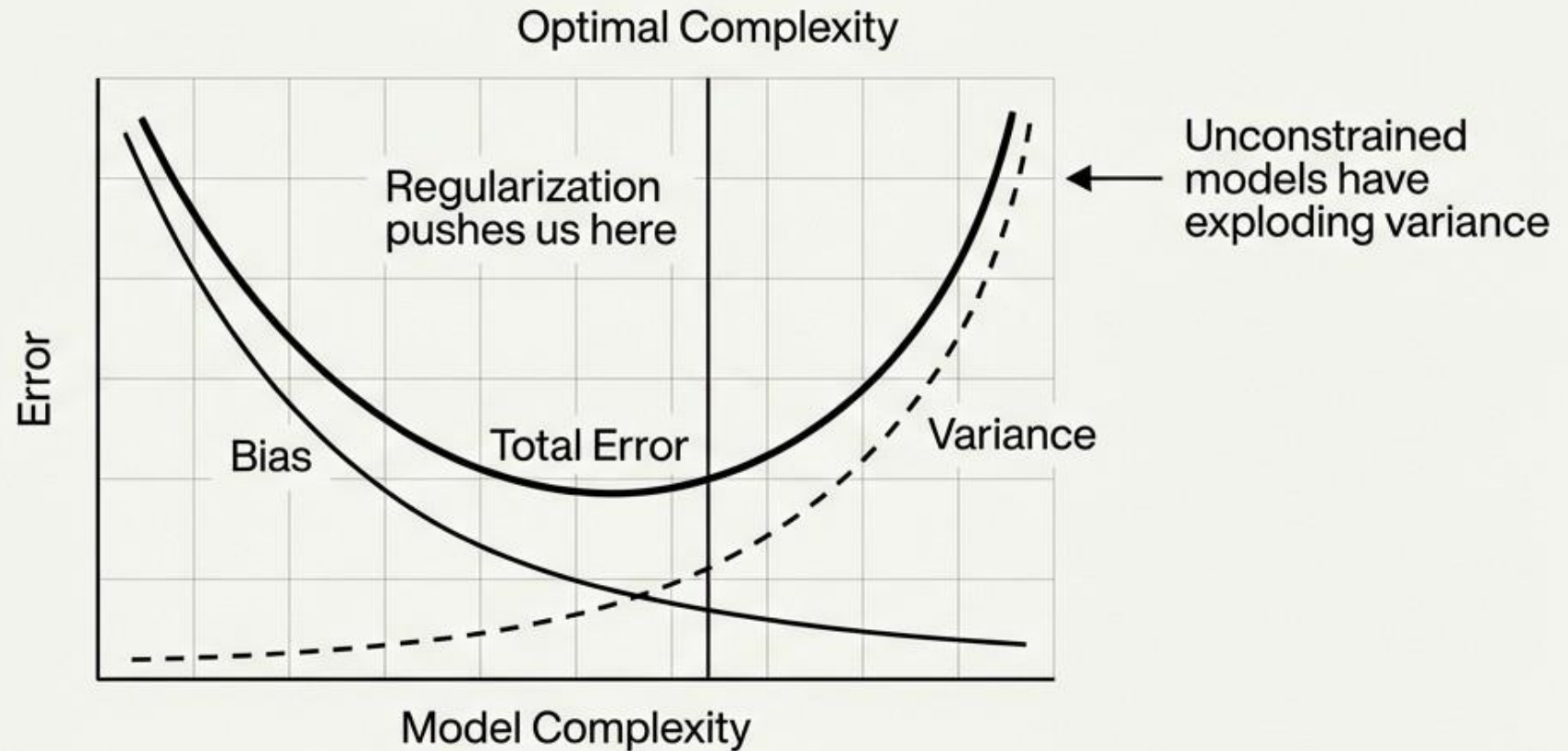
Outcome: Sparse weights (many zeros).

Best For: Feature selection, interpretability.

Verdict: SPECIALIST. Use for high-dimensional sparse data.

Key Trade-off: Both methods reduce variance by deliberately introducing a small amount of bias.

Regularization Is Intentional Bias



We accept a small error in accuracy (Bias) to gain a massive reduction in sensitivity to data perturbations (Variance). The result is lower Total Error.

Derivation

Gradient Descent Update Step

$$J_{L2}(w) = \frac{1}{2n} \|Xw - y\|^2 + \frac{\lambda}{2} \|w\|^2 \quad \|w\|^2 = w^T w$$

$$\nabla J_{L2}(w) = \frac{1}{n} X^T (Xw - y) + \lambda w$$

$$w^{(t+1)} = w^{(t)} - \eta \left(\frac{1}{n} X^T (Xw^{(t)} - y) + \lambda w^{(t)} \right)$$

$$w^{(t+1)} = (1 - \eta\lambda)w^{(t)} - \eta \frac{1}{n} X^T (Xw^{(t)} - y)$$

$$\boxed{w^{(t+1)} = (1 - \eta\lambda)w^{(t)} - \eta \nabla J_{\text{data}}}$$

Normal Equation

$$\frac{1}{n} X^T (Xw - y) + \lambda w = 0$$

$$X^T Xw - X^T y + n\lambda w = 0$$

$$(X^T X + n\lambda I)w = X^T y$$

$$\boxed{w = (X^T X + n\lambda I)^{-1} X^T y}$$

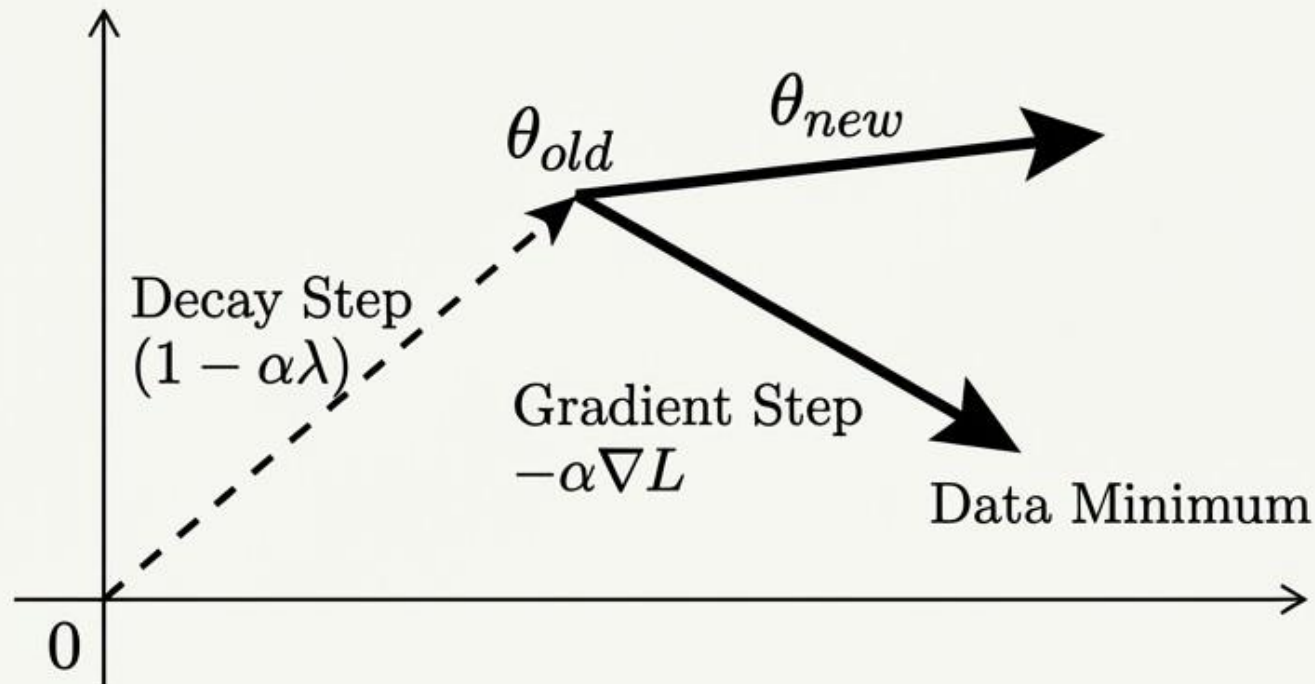
$$X^T X \rightarrow X^T X + n\lambda I$$

Gradient Descent Dynamics: Weight Decay

$$\theta_{new} := (1 - \alpha\lambda)\theta_{old} - \alpha\nabla L$$

Training is a constant state of tension.

1. The Gradient term tries to push parameters to fit the data perfectly.
2. The Decay term shrinks parameters by a percentage at every step.



Result: The model only keeps weights that contribute significantly to reducing error; everything else decays away.

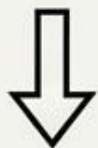
Shaping the Optimization Landscape (Linear Algebra Lens)

The Mathematical Transformation

The Hessian Matrix determines curvature.

$$\mathbf{H} = \mathbf{X}^T \mathbf{X}$$

Ill-conditioned. Singular eigenvalues.
Flat curvature.

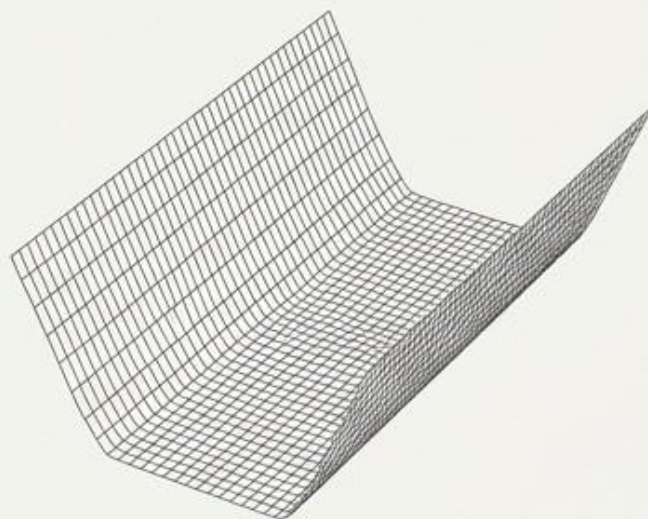


$$\mathbf{H}_{reg} = \mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}$$

Well-conditioned. Invertible.
 $\lambda \mathbf{I}$ adds curvature to all dimensions.

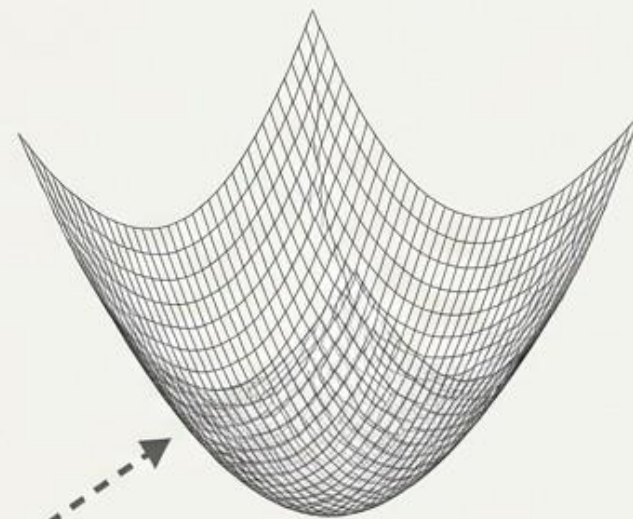
Visualizing the Landscape

Unregularized (Unstable)



Flat Valley

Regularized (Stable)



Bowl

$\lambda \mathbf{I}$ adds curvature to all dimensions.

Matrix Stability: Why We Add λI

The Bug

$$\theta = (X^T X)^{-1} X^T y$$

Bug: If features are collinear, the determinant is 0. This is a Singular Matrix. The code will crash.

The Patch

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

The Stability Patch: Adding values to the diagonal guarantees invertibility.

Regularization isn't just theory; it is a numerical stability patch for your linear algebra engine.

Lab 3: The Overfitting Demo

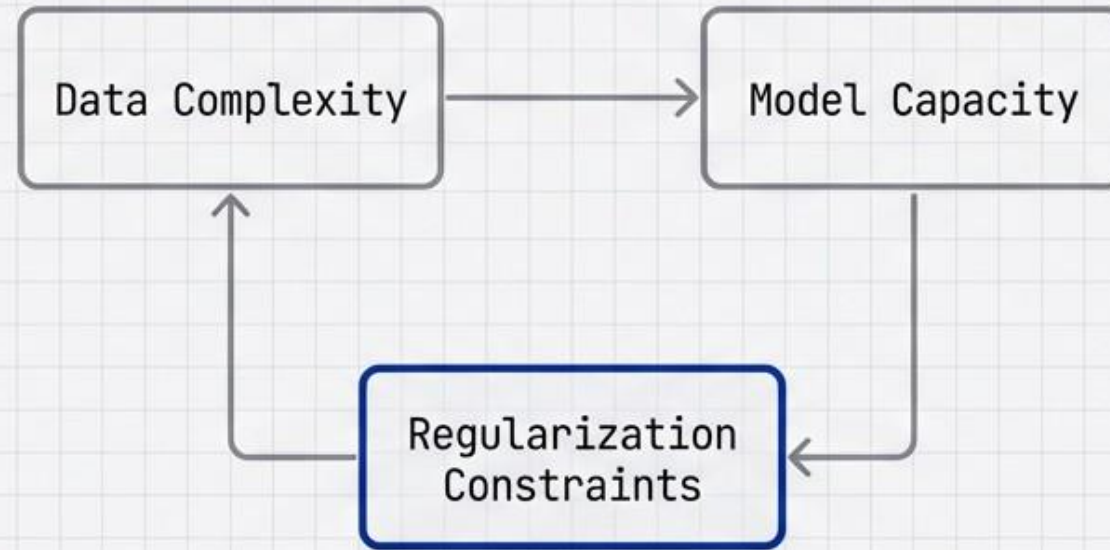
```
[1]: # Task 1: Generate Noisy Data
x = np.linspace(0, 1, 10)
y = np.sin(2 * np.pi * x) + noise
x, noise, y
```

```
[2]: # Task 2: Break the Model
degree = 20
# Fit polynomial... observe explosion
```

```
[3]: # Task 3: The Fix (Manual Ridge)
I = np.eye(degree + 1)
w = np.linalg.inv(X.T @ X + lambda * I) @ X.T @ y
```

- **Objective:** See the variance explosion with your own eyes.
- **Constraint:** No 'sklearn' magic yet.
- **Tool:** Use raw 'numpy.linalg.inv' to implement the closed-form solution.

Machine Learning is System Design



- Optimization minimizes Training Error (The Blue Line).
- Generalization minimizes Test Error (The Red Line).
- Capacity Control bridges the gap.

We do not simply "find patterns." We control what the model is ALLOWED to learn.